

SRS

AI-Powered WhatsApp Business Agent (RAG Engine)

Project Team: Ahmad Bilal Fazal (BSAI24005) & Mah Zarreen Sajjad (BSAI24033)

1. Preface

This document outlines the requirements for the **AI-Powered WhatsApp Business Agent v2.2**. This system leverages **Retrieval-Augmented Generation (RAG)** to provide intelligent, context-aware responses to customer inquiries on WhatsApp. By utilizing **Hugging Face** transformer models and a **Vector Database**, the agent can answer questions grounded specifically in uploaded business documents, avoiding the limitations of standard rule-based bots. This document serves as a complete reference for developers, students, and evaluators.

2. Introduction

2.1 Purpose of the System

The purpose of this system is to develop a production-ready automation engine that can:

1. **Accept Unstructured Data:** Ingest business documents (PDF, TXT, DOCX) as a knowledge base.
2. **Process Natural Language:** Understand user questions sent via WhatsApp without relying on exact keywords.
3. **Generate Grounded Answers:** Use a RAG pipeline to retrieve relevant facts and generate a response with citations to the source document.

2.2 Scope of the System

The system will:

- **Web-Based Management:** Provide a **Gradio/Streamlit** GUI for administrators to upload files and view chat analytics.
- **WhatsApp Integration:** Use the **Twilio API** to bridge the gap between the AI backend and the mobile user.
- **AI Stack:** Utilize **Hugging Face** for embeddings (**sentence-transformers**) and response generation (**BART/T5**).
- **Secure Tunneling:** Employ **ngrok** to allow the local Flask server to receive real-time webhooks from WhatsApp.

The system will **not** support real-time video processing or multi-language translation in this version.

2.3 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation: An AI framework that retrieves data from an external knowledge base to ground LLM responses.
LLM	Large Language Model: A deep learning model trained to understand and generate text.
Embeddings	Vector representations of text used to measure semantic similarity.
Webhook	A URL that receives real-time data from an external service (Twilio).

3. User Requirements Definition

3.1 Functional Requirements

1. **Document Ingestion:** The system shall allow users to upload PDF, DOCX, and TXT files to build the AI's knowledge.
2. **Natural Language Querying:** The system shall allow end-users to input questions in plain English via WhatsApp.
3. **Contextual Response:** The system shall generate an answer based strictly on the uploaded content.
4. **Source Citations:** The system shall display which document or page the information was retrieved from.
5. **Session Management:** The admin shall be able to view and delete past chat history.

3.2 Non-Functional Requirements

1. **Usability:** The management dashboard shall have a simple, intuitive interface for non-technical business owners.
2. **Performance:** The response time for a WhatsApp message shall be ≤ 12 seconds.
3. **Accuracy:** The system shall enforce strict hallucination control, only answering if the data exists in the knowledge base.
4. **Security:** The system shall secure its API endpoints and use environmental variables for sensitive keys.

4. System Architecture

The system is architected as a set of integrated microservices:

- **Frontend:** Gradio/Streamlit application for the admin dashboard.
- **Backend:** Flask/FastAPI server handling the RAG pipeline and Webhooks.
- **Vector Database:** **FAISS** or **ChromaDB** for storing document embeddings.
- **External APIs:** Twilio (for WhatsApp) and Hugging Face (for ML models).

5. System Requirements Specification

5.1 Functional System Requirements

4.1.1 Document Input Handling (Requirement ID: F-01)

- **Description:** The system shall process uploaded files into the knowledge base.
- **Input:** Files (PDF, DOCX, TXT).
- **Output:** Text is extracted, chunked, and stored as embeddings.
- **Error Handling:** If an unsupported file is detected, display: *"Invalid format or corrupted file"*.

4.1.2 WhatsApp Query Input (Requirement ID: F-02)

- **Description:** The system shall handle incoming text from the Twilio Webhook.
- **Input:** Text-based question from a mobile device.
- **Constraints:** Multilingual support for standard customer queries.

4.1.3 Answer Generation (Requirement ID: F-03)

- **Processing Steps:** 1. Convert query to vector embedding. 2. Retrieve top-3 relevant document chunks. 3. Generate response using Hugging Face model.
- **Error Handling:** If no info is found, state: *"Limited Information Available"*.

5.2 Non-Functional System Requirements

- **Retrieval Speed:** Semantic search must complete in ≤ 1 second.
- **Compatibility:** The application must run on any system supporting **Docker** or **Python 3.x**.

6. System Models (Use Cases)

6.1 Asking a Contextual Question via WhatsApp

- **Primary Actor:** User (Customer).
- **Supporting Actor:** RAG Pipeline.
- **Basic Flow:**
 1. User asks: *"What is your return policy?"* via WhatsApp.
 2. System retrieves chunks from [return_policy.pdf](#).
 3. System generates: *"You can return items within 30 days..."*.

4. Answer is sent back via Twilio with citation.

6.2 Handling an Invalid File Upload

- **Preconditions:** User uploads an unsupported format (e.g., `.exe` or `.zip`).
- **Flow:** System detects invalid content and displays an error message.
- **Postcondition:** Knowledge base remains clean.

Task 3: Requirements Elicitation and Analysis

1. Elicitation Technique

To discover and refine the requirements for the AI-Powered WhatsApp Business Agent, we utilized a combination of **Analysis of Existing Systems** and **Brainstorming**.

- **Analysis of Existing Systems:** We analyzed current rule-based WhatsApp automation tools (like basic Twilio Auto-responders) and identified a major "pain point": they fail to understand natural language or context, often leading to customer frustration when exact keywords are not used.
- **Brainstorming:** Our team (Ahmad Bilal Fazal and Mah Zarreen Sajjad) conducted brainstorming sessions to determine how to integrate **Retrieval-Augmented Generation (RAG)** into a mobile interface. We focused on how a user could upload a PDF and have the AI "read" it to provide answers, which led to the requirement for a Vector Database and a Generative ML model.
- **Appropriateness for AI Use Case:** These methods were appropriate because RAG is a technical solution to a specific problem—information retrieval. Analyzing existing failures allowed us to define the "Complexity" required for the project, ensuring we moved beyond simple dictionary mapping to a true AI-driven logic layer.

2. Stakeholder Identification

We have identified two primary stakeholders for this system, each with specific needs and "pain points":

- **Primary Stakeholder: Small Business Owners (Administrators)**
 - **Pain Point:** They spend excessive time answering the same questions (pricing, hours, policies) over WhatsApp.
 - **How the ML Model Addresses It:** The RAG pipeline allows them to simply upload their existing business documents (PDF/DOCX), and the ML model handles the inquiries automatically based on that data.
- **Secondary Stakeholder: End-Customers (Users)**
 - **Pain Point:** Customers often receive generic, unhelpful responses from traditional bots or have to wait hours for a human agent.

- **How the ML Model Addresses It:** By using a transformer-based LLM (like GPT-4o or BART), the customer receives a natural, contextually accurate answer in seconds, improving the overall user experience.